

Introduction to Greedy Algorithms

1. What is a Greedy Algorithm?

A **greedy algorithm** builds up a solution piece by piece, always choosing the next piece that offers the most immediate benefit. It does not reconsider its choices, which can make it very efficient, though not always optimal for all problems.

Key idea: *Make the locally optimal choice at each step, hoping it leads to a globally optimal solution.*

Common examples include:

- Activity (interval) scheduling
- Huffman coding
- Dijkstra's shortest path algorithm
- Fractional knapsack problem

Here are some fun examples regarding real-life *greedy* approaches.

- Choosing Snacks at a Party
Greedy move: You grab the tastiest-looking item immediately.
Why it's greedy: You're making a local (tasty) choice without thinking about whether you'll be too full for better snacks later.
- Choosing the Shortest Line at a Grocery Store
Scenario: You pick the line with the fewest people.
Greedy move: "This line is shortest, must be fastest."
But: Someone in front has a cart full of items.
Realization: Greedy doesn't always check deeper details!

2. Case Study: Interval Scheduling

Problem Statement

Given a set of n activities (or intervals), each with a **start time** s_i and **finish time** f_i , schedule as many non-overlapping activities as possible.

Input: A set of intervals $(s_1, f_1), (s_2, f_2), \dots, (s_n, f_n)$

Goal: Select the largest possible subset of non-overlapping intervals.

Example Intervals

Activity	Start Time	Finish Time
A	1	4
B	3	5
C	0	6
D	5	7
E	3	9
F	5	9
G	6	10
H	8	11
I	8	12
J	2	14
K	12	16

Greedy Strategy: Earliest Finish Time First

1. Sort intervals by finish time.
2. Initialize result set to empty.
3. Iterate through the sorted intervals:
 - If an interval starts after or when the last selected one ends, select it.

Sorted Intervals by Finish Time

(1, 4), (3, 5), (0, 6), (5, 7), (3, 9), (5, 9), (6, 10), (8, 11), (8, 12), (2, 14), (12, 16)

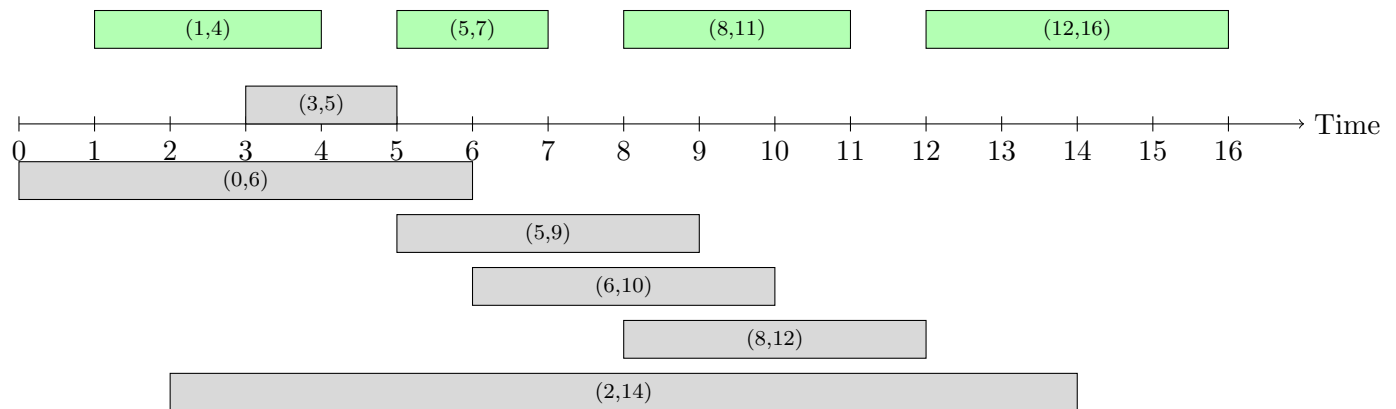
Step-by-step Selection

- Select (1,4)
- Skip (3,5), (0,6)
- Select (5,7)
- Skip (3,9), (5,9), (6,10)
- Select (8,11)
- Skip (8,12), (2,14)
- Select (12,16)

Final intervals:

(1, 4), (5, 7), (8, 11), (12, 16)

Timeline Diagram



3. Greedy Algorithm: Formal Pseudocode

Algorithm 1 GreedyIntervalScheduling

Require: Set of activities with start and finish times

Ensure: Maximum set of mutually compatible activities

```

1: Sort activities by increasing finish time
2: Let  $A \leftarrow \emptyset$ 
3: Let  $lastFinish \leftarrow 0$ 
4: for each activity  $i$  in sorted order do
5:   if  $start_i \geq lastFinish$  then
6:     Add activity  $i$  to  $A$ 
7:      $lastFinish \leftarrow finish_i$ 
8:   end if
9: end for
10: return  $A$ 

```

4. Why This Greedy Approach Works

This strategy works because choosing the earliest finishing activity:

- Leaves as much time as possible for future activities
- Prevents longer activities from blocking shorter ones later

This problem satisfies the **greedy-choice property** and **optimal substructure**, making it a perfect fit for greedy algorithms.

5. Summary

Greedy algorithms work well when:

- The problem has optimal substructure
- A locally optimal choice leads to a globally optimal solution

Interval Scheduling is a classic example where a greedy algorithm guarantees an optimal result.